軟體工程

第三課：需求工程 (Requirements Engineering)

授課教師：薛念林 教授
資訊工程學系
逢甲大學

第三章：課程大綱與核心架構

第一部分：需求基礎、分類與度量指標

- **3.1 需求工程基礎：** 何謂需求工程、延遲修復缺陷的代價曲線、使用者需求 vs. 系統需求。
- **3.2 利害關係人與需求分類：** 利害關係人多重視角、功能性需求與領域規則。
- **3.3 非功能性需求與量化指標：** 品質屬性 (FURPS+)、主觀目標轉化為可客觀測試的量化指標。

第二部分：需求擷取、驗證與 AI 賦能

- **3.4 需求擷取與系統建模：** 四大擷取步驟、訪談與情境分析、UML 使用案例圖與規格書撰寫。
- **3.5 需求驗證與變更管理：** 五大驗證支柱（有效性、一致性、完整性等）、追溯矩陣與變更控制。
- **3.6 AI 賦能需求工程：** 大型語言模型輔助、使用者故事生成、模糊性檢測與人機協同防護。
- **3.7 觀念總結與經典文獻：** 重點填空測驗、深度思考問題、核心回顧與參考文獻。

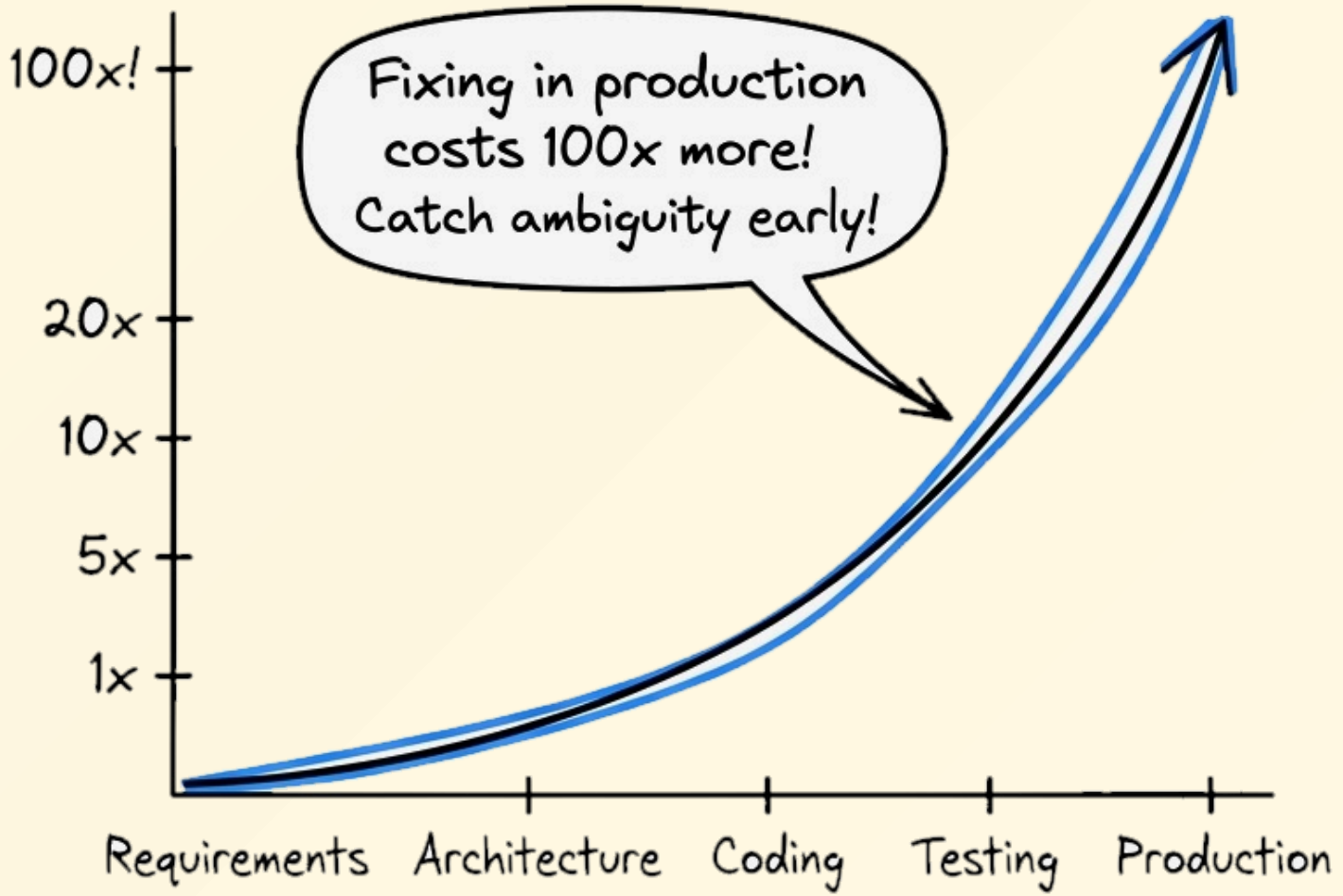
本章核心思考問題

- 何謂**需求工程 (Requirements Engineering, RE)**？為何在早期捕捉並修正需求錯誤如此關鍵？
- **使用者需求 (User Requirements)** 與**系統需求 (System Requirements)** 的本質差異為何？
- **功能性需求 (Functional)**、**非功能性需求 (Non-Functional)** 與**領域需求 (Domain Requirements)** 如何區分？
- **需求擷取與分析 (Elicitation & Analysis)** 流程的四大核心活動為何？
- 如何將模糊主觀的非功能品質目標，轉化為**可客觀量化與測試的工程指標**？
- 現代**大型語言模型 (LLM)** 如何加速需求工程流程？人機協同 (Human-in-the-Loop) 的防護機制又該如何落實？

3.1 需求工程基礎 (Foundations of RE)

> *"The hardest single part of building a software system is deciding precisely what to build."*
> *「建構軟體系統最困難的單一工作，就是精確決定到底要打造什麼。」*
> *— Fred Brooks, No Silver Bullet (人月神話作者, 1987)*

Exponential Cost of Requirements Errors



需求工程的基礎本質 (Foundations of RE)

> *建立客戶對系統所需之服務，以及系統運作與開發過程中所受限制的系統化工程程序。*

- **需求的雙重商業功能 (Dual Function):**
  1. **招標與合約洽談 (Contract Bidding):** 高階抽象敘述，容許不同承包廠商提出多元解法與報價。
  2. **合約實施基準 (Contract Basis):** 精確詳盡的功能規格，嚴格定義團隊最終交付的具體技術內容。
- **工程的核心使命 (The Fundamental Goal):**
  - 跨越利害關係人複雜、模稜兩可且充滿變數的人類真實世界，與確定、精準且邏輯嚴密的軟體系統之間的溝通鴻溝。

User Requirement (Customer Intent)

The MHC-PMS shall generate monthly summary reports of drug costs prescribed by clinics

Engineering Translation



System Requirements (Developer Contract)

- 1.1: Triggered on last working day of month
- 1.2: Standard layout: drug name, doses, total cost
- 1.3: Handles 10k records < 60s without DB lock

需求層次：使用者、系統與領域需求

- **使用者需求 (User Requirements):**
  - 以自然語言搭配簡明圖表撰寫的高階抽象陳述。
  - 目標受眾：**客戶端主管、系統終端使用者、專案經理與非技術利害關係人。**
  - 核心焦點：從使用者視角界定系統應提供**什麼 (What)** 服務。
- **系統需求 (System Requirements):**
  - 結構化、技術精確的詳細規格，嚴格定義系統功能、服務與運作約束。
  - 目標受眾：**軟體架構師、後端與前端工程師、品質驗證 (QA) 測試人員。**
  - 核心焦點：作為系統具體實作、驗收測試與工程合約的技術基線 (Technical Contract)。
- **領域需求 (Domain Requirements):**
  - 源自業務領域、產業法規或物理環境的根本運作約束 (如醫療隱私法規、金融合規)。
  - 不可撼動性：直接反映現實世界的強制規範，往往具備凌駕使用者偏好的最高約束力。

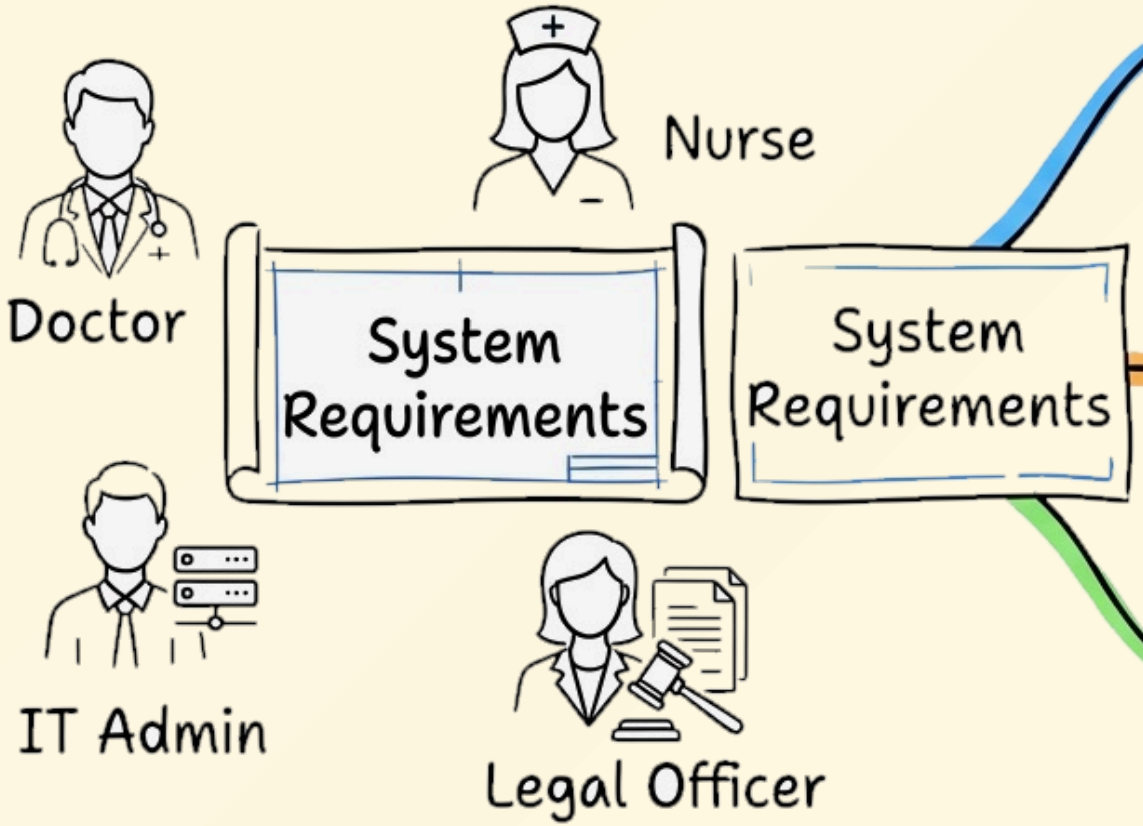
使用者需求 vs. 系統需求範例 (MHC-PMS 心理健康管理系統)

- **使用者需求 (高階業務意圖):**

> *「心理健康病患管理系統 (MHC-PMS) 應每月產出彙總報表，顯示各診所開立處方藥品之總花費金額。」*

- **系統需求 (精確技術合約規格):**
  - **1.1：** 系統應於每月最後一個工作日的 23:59:59，自動產出各註冊診所當月藥品成本的彙總報表。
  - **1.2：** 報表應採用標準版型呈現，明確列出藥品代碼、藥品名稱、開立總劑量單位與各別加總總金額。
  - **1.3：** 若單一診所藥品資料量超過 10,000 筆，報表產製計算應於 60 秒內完成，且不得造成線上交易資料庫鎖定 (Locking)。
- **領域需求 (醫療法規與臨床約束):**
  - **D.1 (病患隱私去識別化)：** 依據醫療資訊隱私法規 (如 HIPAA / 醫療法)，費用報表必須高度彙總，嚴禁反推或洩漏個別病患身分。
  - **D.2 (法定藥典標準代碼)：** 藥品代碼與計量單位必須嚴格遵循國家法定藥典標準 (如健保代碼 / FDA NDC)，禁止自訂非正規別名。

The Three Types of Software Requirements



Functional Requirements:
Specific services, behaviors, calculations, user actions

Non-Functional Requirements:
Quality attributes, response time, security, 99.9% uptime

Domain Requirements:
Industry laws, healthcare compliance, physical constraints

觀念檢測：使用者需求 vs. 系統需求 (CCQ 1)

在需求工程中，「使用者需求 (User Requirements)」與「系統需求 (System Requirements)」最關鍵的實務運作差異為何？

- **A.** 使用者需求是利害關係人的高階目標；系統需求則是供開發者實作的精確功能合約。
- **B.** 使用者需求限定於 UI 線框圖；系統需求則專指後端資料庫的資料表綱要設計。
- **C.** 使用者需求簽署後絕不可變更；系統需求則需在每日站立會議中持續進行重構。
- **D.** 使用者需求來自外部法規稽核團隊；系統需求則由編譯器工具自動解析產生。



3.2 系統利害關係人與需求分類 (Stakeholders & Classification)

> *"Software systems have many masters. If you fail to identify who has a stake, their unspoken expectations will become your project's failure."*
> *「軟體系統往往有多位主人。若未能全面辨識誰有利害關係，他們未曾言明的隱藏期望，終將成為專案失敗的致命傷。」*

系統利害關係人 (System Stakeholders)

> *任何受到系統運作所影響，或對系統行為與產出結果具備合理正當利益關係的個人或組織。*

- **利害關係人多元類別 (Types of Stakeholders):**
  - **終端操作者 (End Users):** 病患、主治醫師、護理師、門診掛號櫃檯行政人員。
  - **系統維運與管理層 (System Managers):** 醫院診所主管、資訊長 (CIO)、資訊安全與法規合規主管。
  - **外部利害關係人 (External Stakeholders):** 衛生主管機關、醫療器材認證審查機構、健保給付與商業保險機構。
- **利害關係人目標衝突之挑戰 (The Stakeholder Conflict Challenge):**
  - 不同角色常具備相互競爭甚至矛盾的需求（例如：醫師要求單鍵快速開藥無摩擦；資安主管要求強制多因子身分認證與不可竄改稽核日誌）。

實務案例研討：UberEats 的利害關係人生態系

> *像 UberEats 這類現代外送平台，生動展現了需求如何從具備相互競爭利益的多邊利害關係人生態系中湧現。*

- **多邊利害關係人生態系 (Multi-Sided Stakeholder Ecosystem):**
  - **飢腸轆轆的消費者 (Eater):** 要求極速外送、精準預估到達時間 (ETA)、即時 GPS 地圖動態與無障礙退款機制。
  - **合作餐廳店家 (Restaurant Merchant):** 要求充裕的廚房備餐緩衝時間、外送員準時抵達，且一旦起鍋烹調便不可隨意取消扣款。
  - **外送員夥伴 (Delivery Courier):** 要求公平的派單演算法、透明公開的小費分潤、合理的行車距離與安全的違停容許時間。
  - **平台營運核心 (Uber Platform):** 最佳化雙邊市場流動性、透過尖峰動態加價平衡供需，並嚴密防範優惠碼刷單詐欺。
  - **外部監管機關 (External Regulators):** 強制要求食品保溫衛生檢驗標準、外送勞動工時安全保障，以及開立法定統一發票。
- **利害關係人衝突案例 (The Stakeholder Conflict Challenge):**
  - *消費者 vs. 餐廳：* 消費者於下單 5 分鐘後取消；餐廳已使用高檔食材下鍋烹調。究竟這筆食材與營運損失該由誰吸收？

UberEats 實務剖析：需求三大支柱落地

> *需求三大支柱平衡了使用者服務、運作品質與環境法規遵循。*

1. 功能性需求

系統服務與主動行為

- 依飲食限制 (素食/無麩質) 與即時外送預估時間 (ETA) 篩選餐廳。
- 依外送員即時 GPS 座標與交通工具智慧派發最佳外送任務。
- 訂單完成交付後自動執行三方分潤結算與款項撥付。

2. 非功能性需求

品質屬性與架構約束

- **效能表現：** 外送員 GPS 座標在追蹤地圖上更新延遲 ≤ 2 秒。
- **高可用性：** 晚餐尖峰用餐時段平台可用性維持在 $\geq 99.99\%$。
- **資訊安全：** 信用卡資訊處理嚴格符合 PCI-DSS Level 1 規範。

3. 領域需求

外部環境法規強制約束

- **食品安全：** 熟食運送時間與保溫嚴格符合地方衛生局法規。
- **稅務合規：** 即時開立具備法定效力的電子統一發票。
- **勞動法規：** 嚴格遵循外送員連續上工時數與排班休息上限。

功能性需求面臨之挑戰：模糊、完全性與一致性

- **需求陳述模糊性 (Requirements Imprecision):**
  - 自然語言先天的語意多義性極易造成多方利害關係人的嚴重認知衝突。
  - *UberEats 實務範例：* 「消費者可在餐點送達前取消訂單。」
    - *消費者認知：* 只要外送員還沒抵達門前，我隨時都能按取消並拿回全額退款。
    - *餐廳店家認知：* 一旦廚房開始備料下鍋烹調，該筆訂單就絕對不可取消或退款！
- **完全性 vs. 一致性 (Completeness vs. Consistency):**
  - **完全性 (Completeness):** 必須窮舉所有極端邊界情境與例外服務（例如：外送員途中車輛拋錨、暴雨缺單時之自動備料替換邏輯）。
  - **一致性 (Consistency):** 規格條文間絕不可存在互相矛盾衝突的邏輯（例如：「承諾消費者可無條件取消退款」 vs. 「保障店家備餐後獲得全額補償撥款」）。
  - *殘酷工程現實：* 在大型多邊平台上，企圖在專案一開始就達到 100% 完全性與絕對一致性是不切實際的，必須仰賴疊代探索與持續精煉。

觀念檢測：領域需求分類辨識 (CCQ 2)

請參考 UberEats 外送平台的這段需求規格說明：

> *「因應市政食品衛生安全法規，易腐熟食的外送運送時間不得超過 45 分鐘，且運送容器全程須維持在 60°C 以上。」*

請問上述敘述屬於哪一種軟體需求類別？

- **A.** 領域需求 (Domain Requirement，由運作環境、產業法規或物理定律所施加的強制約束)
- **B.** 功能性需求 (Functional Requirement，指定系統必須執行的主動計算、特定功能或服務)
- **C.** 非功能性需求 (Non-Functional Requirement，針對效能、擴充性或可用性等整體系統品質特性)
- **D.** 使用者需求 (User Requirement，由終端消費者所提出之高階自然語言願景或業務目標)

課堂活動：Cybercab 需求三元組腦力激盪 (Pair Discussion)

> *情境設定：假設你是 Elon Musk，你的無人自駕車 **Cybercab** 已經合法上路。平時除了自己代步開以外，閒置時還能加入自動駕駛車隊租給其他人搭乘賺取收益。*

1. 功能需求 (Functional)

系統提供的具體功能與業務服務：

- **引導思考面向：**
  - 乘客如何透過 App 進行遠端叫車、定位追蹤與感應上車？
  - 車主如何將車輛一鍵切換釋出給自動駕駛車隊營運？
  - 平台如何自動完成行程計費，並將租金收益分潤給車主？

2. 非功能需求 (NFR)

系統運行的品質屬性與效能約束：

- **引導思考面向：**
  - 車輛在即時感測避障與緊急煞車上的毫秒級反應時間極限？
  - 全球自駕車派遣雲端服務必須具備多高的可用性 (Uptime)？
  - 面對遠端駭客劫持威脅，車載網路需要何種資安防護？

3. 領域需求 (Domain)

交通法規、安全認證與環境強制約束：

- **引導思考面向：**
  - 無人駕駛車輛必須符合哪些國家級自駕安全法規 (如 NHTSA)？
  - 發生交通事故時，法規強制要求記錄哪些關鍵數據 (黑盒子/EDR)？
  - 將自用車投入商業共享載客時，法律強制要求投保哪些保險？

3.3 非功能性需求 (NFR) 與量化指標

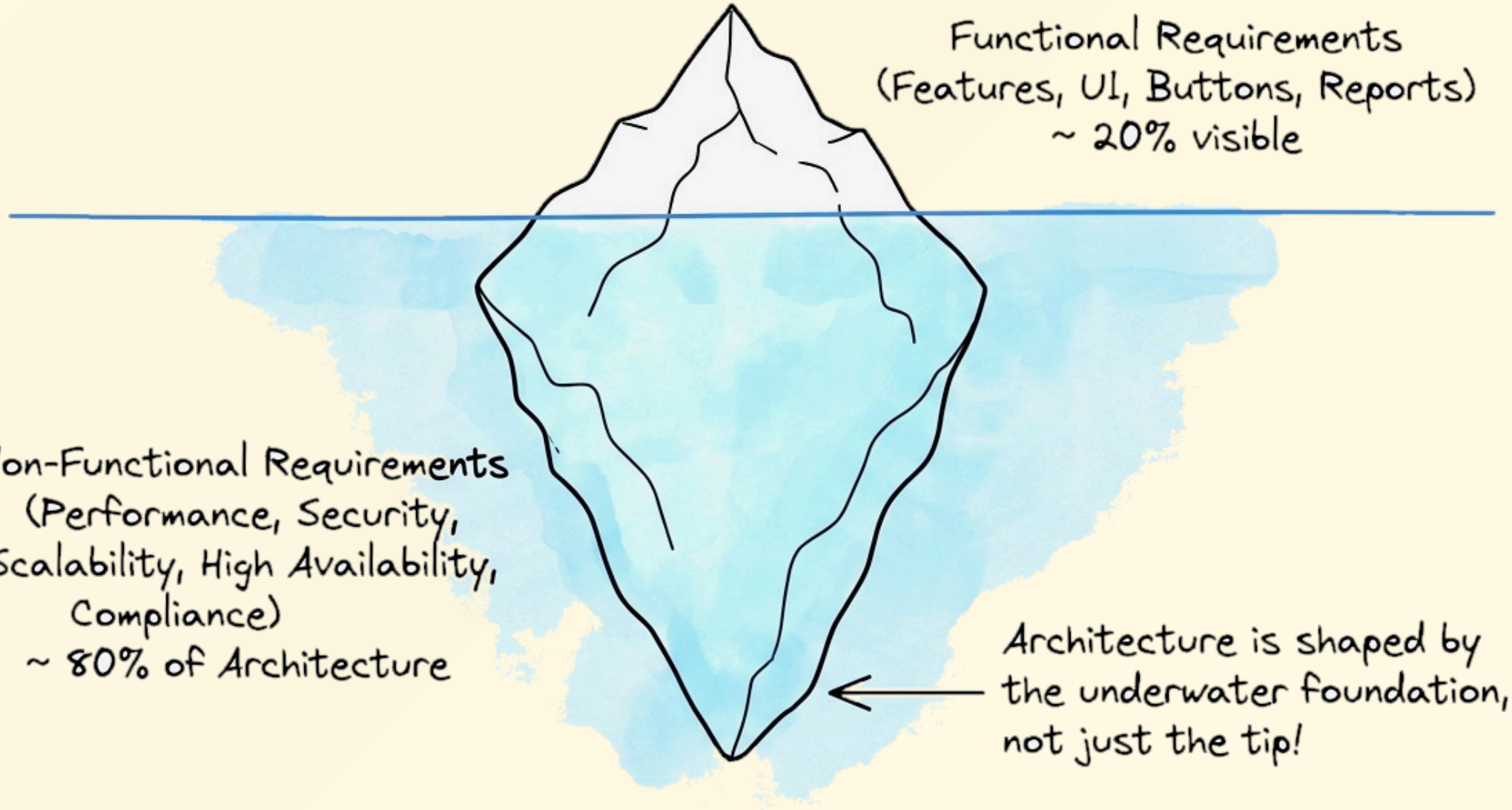
> *"If you can't measure it, you can't manage it — and you certainly cannot verify it."*
> *「如果你無法度量它，你就無法管理它——更絕對無法對它進行客觀驗證。」*
> *— Tom DeMarco (軟體工程度量大師)*

非功能性需求與量化指標 (NFR & Metrics)

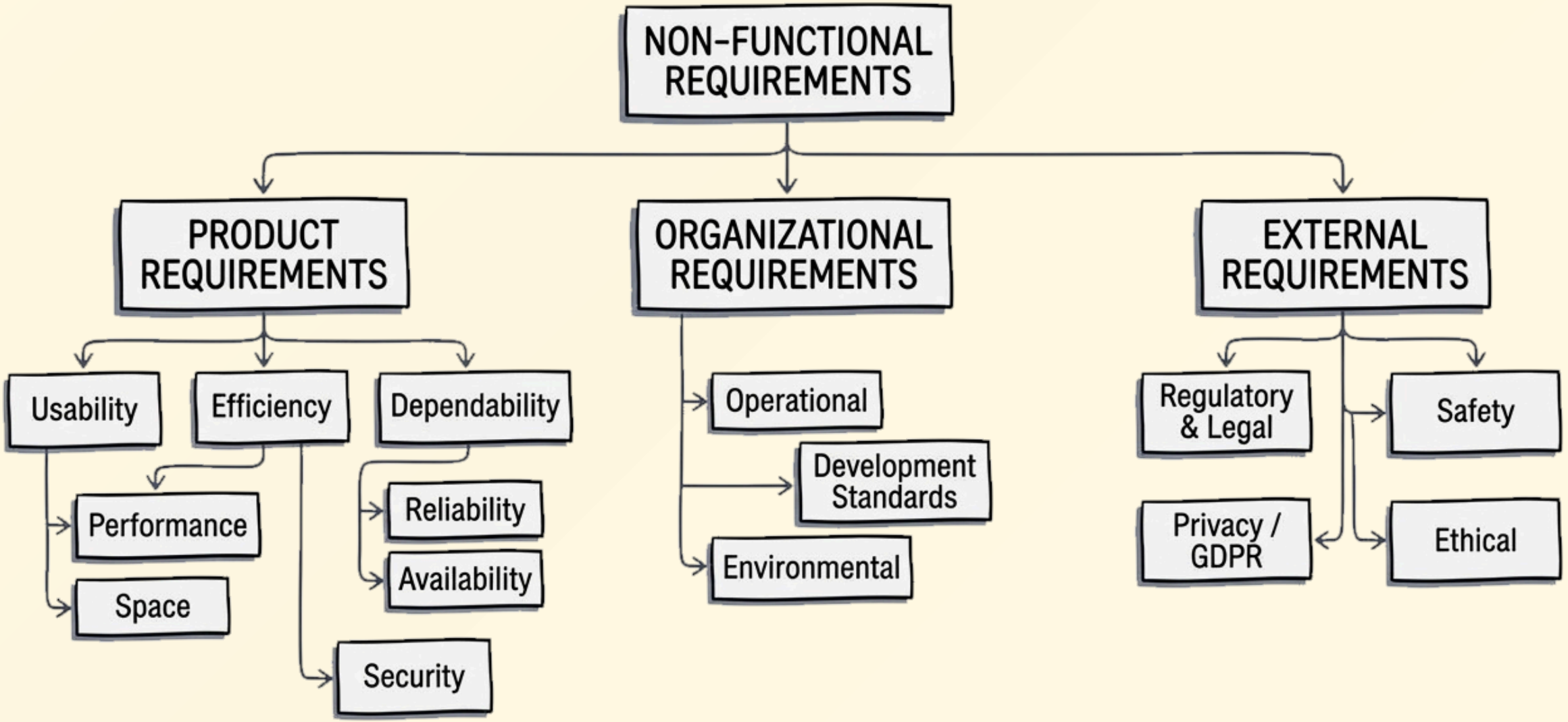
> *對系統所提供服務或功能之約束條件，包括時間限制、安全隱私、可靠度與標準遵循，從根本上塑造軟體架構。*
>
> ”

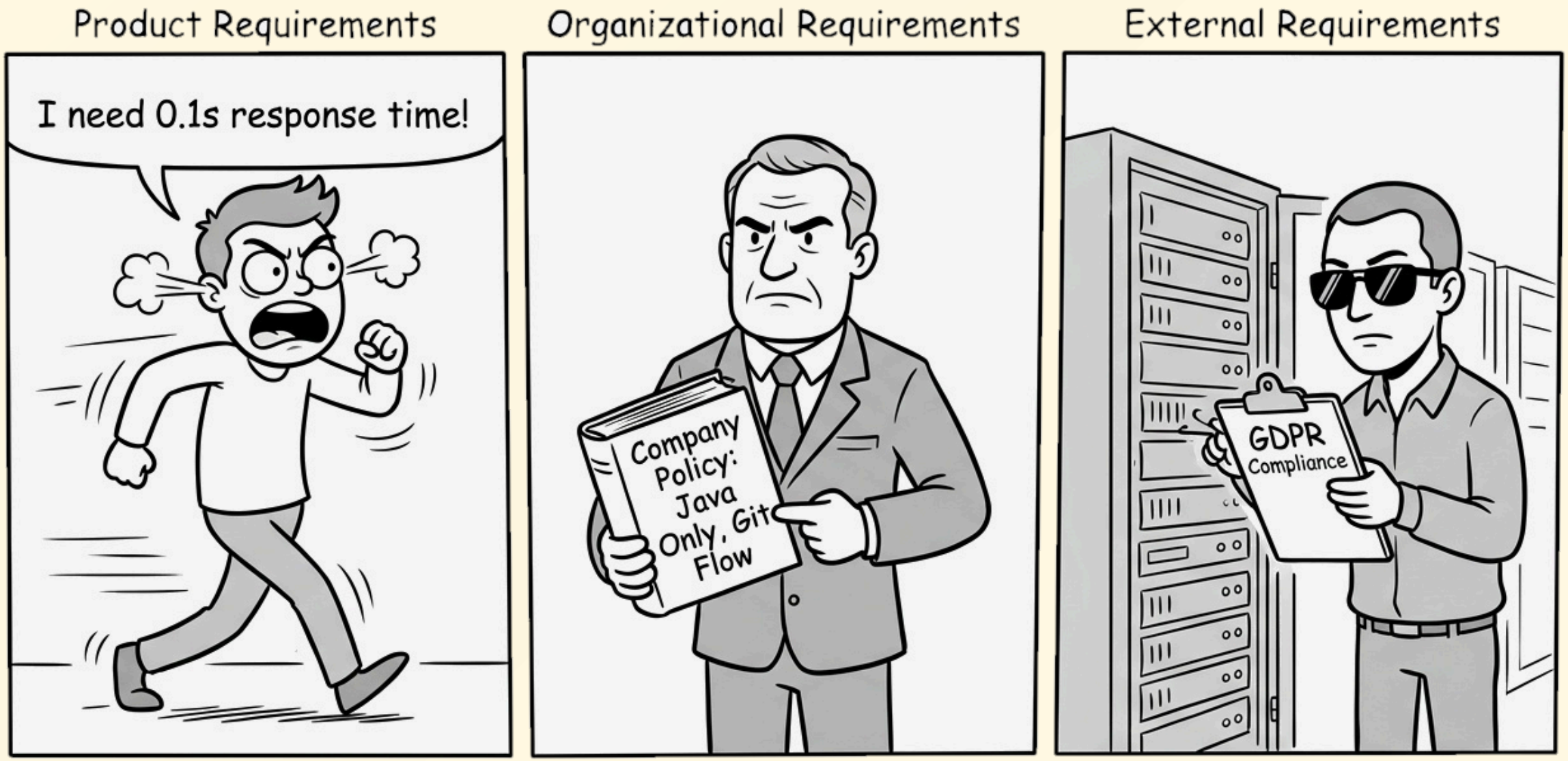
- **全系統級的深遠影響 (System-Wide Impact):**
  - 非功能性需求通常作用於系統**整體架構**，而非單一孤立的功能特性。
  - 未能滿足非功能性需求，足以導致整套系統徹底癱瘓或遭使用者唾棄（例如：反應緩慢或資安漏洞百出的醫療資料庫）。
- **對軟體架構的主導力量 (Architectural Influence):**
  - 效能限制決定了模組間的通訊機制與快取策略；安全合規要求催生了獨立的身分認證中介架構。
  - *深刻工程真理：軟體架構本質上是由非功能性需求所雕刻而成的，其影響力遠超越單純的功能特性！*

Non-Functional Requirements Iceberg



NON-FUNCTIONAL REQUIREMENTS (NFR) HIERARCHY




Product Requirements
I need 0.1s response time!
Organizational Requirements
Company Policy: Java Only, Git Flow
External Requirements
GDPR Compliance

非功能性需求三大核心家族 (Classification of NFRs)

> *非功能性需求源自軟體產品的執行期品質、內部組織的治理規範，以及外部運行環境的法律約束。*

1. 產品需求 (Product)

定義： 規範交付的軟體產品在執行期必須具備的操作行為與運行品質。

- **範疇：** 系統執行效能、可靠度、資訊安全與終端使用者操作體驗。
- **子類別：** 易用性 (Usability)、效率 (Efficiency)、可靠性、安全性。
- **實例：** 查詢回應 ≤ 2 秒、可用性 $\geq 99.99\%$、AES-256 加密。

2. 組織需求 (Organizational)

定義： 源自客戶或開發組織內部管理政策、作業程序與工程標準的要求。

- **範疇：** 開發流程規範、維運治理要求與企業既有基礎架構限制。
- **子類別：** 開發標準 (流程/語言)、維運程序 (備份/災難復原)、平台約束。
- **實例：** 遵循 Git Flow 與 CI/CD 門禁、每日自動異地備份、限定 AWS。

3. 外部需求 (External)

定義： 源自系統及其工程開發流程外部之法規、環境與社會因素之要求。

- **範疇：** 國家法規合規性、主管機關監管標準、生命財產安全與社會倫理。
- **子類別：** 法定與監管規範、人身安全防護 (Fail-Safe)、隱私倫理。
- **實例：** 符合歐盟 GDPR 個資法、政府電子發票規範、醫療防呆機制。

主觀目標 vs. 可驗證非功能需求 (Goals vs. Verifiable NFRs)

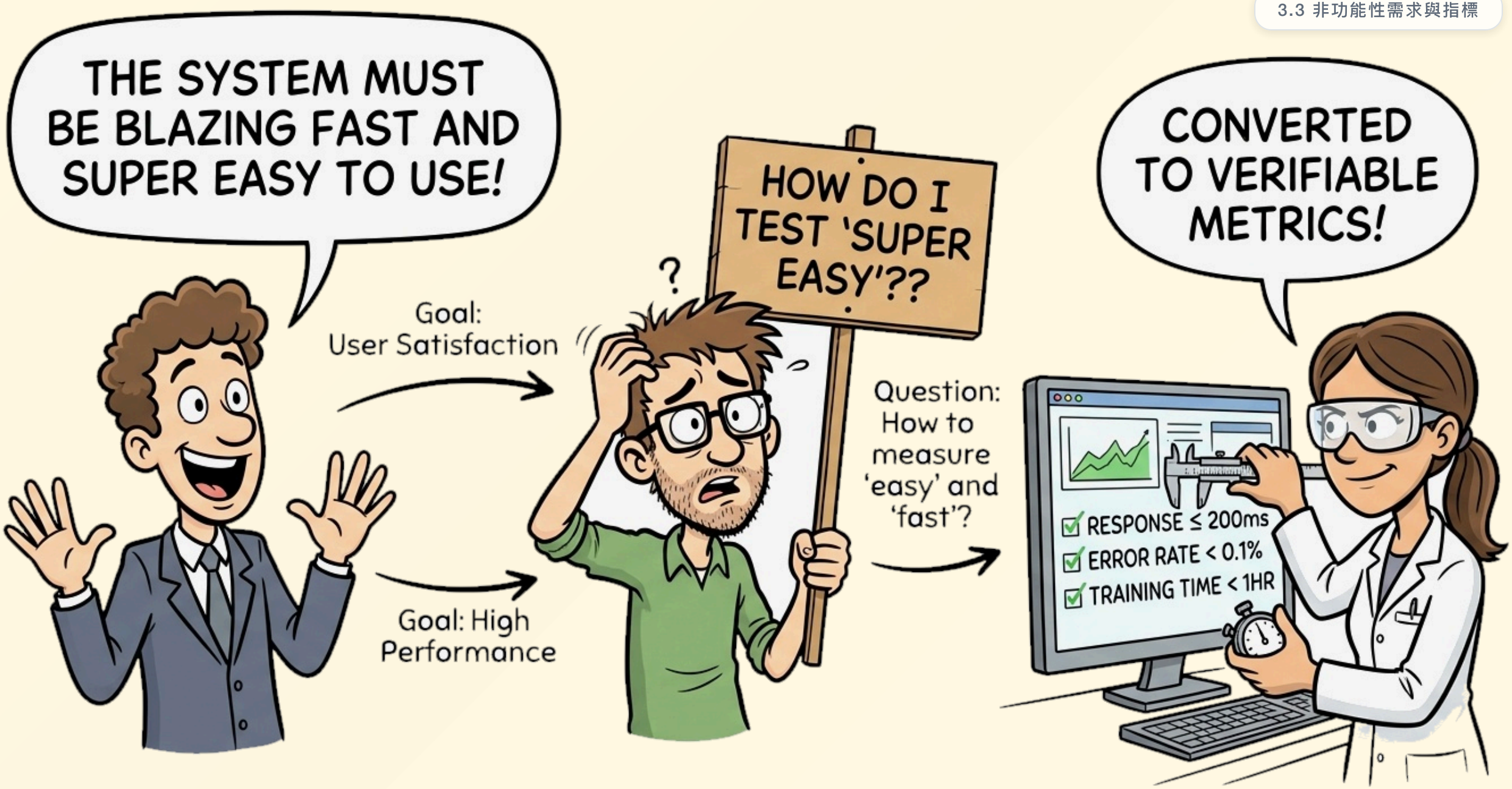
- **主觀目標 (模糊難以測量的願景):**

> *「系統應易於醫護人員操作使用，並大幅降低人為操作錯誤。」*

- **可驗證需求 (具備客觀測試量化標準):**

> *「具備基本電腦常識的醫護人員，在接受 2 小時標準培訓後，即能獨立操作所有核心功能；上線運作後，每位操作員平均每工作日之人為錯誤次數不得超過 2 次。」*

- **工程黃金法則 (The Golden Engineering Principle):**
  - 軟體工程師必須主動將利害關係人主觀抽象的期待，轉化為**可客觀量化、可嚴謹測試驗收的具體指標！**


THE SYSTEM MUST BE BLAZING FAST AND SUPER EASY TO USE!
Goal: User Satisfaction
Goal: High Performance
?
HOW DO I TEST 'SUPER EASY'??
Question: How to measure 'easy' and 'fast'?
CONVERTED TO VERIFIABLE METRICS!
RESPONSE ≤ 200ms
ERROR RATE < 0.1%
TRAINING TIME < 1HR

非功能性需求標準量化指標 (Metrics for Specifying NFRs)

系統特徵 (Property)	可度量指標範例 (Measurable Metric Examples)
運算速度 (Speed)	每秒處理交易數 (TPS)；第 95/99 百分位回應延遲；畫面每秒更新率 (FPS)
儲存容量 (Size)	隨機存取記憶體 (RAM) 占用容量；安裝套件快閃儲存空間 (ROM/Disk)
易用性 (Ease of Use)	達到合格操作所需的教育訓練時數；每週進線客服求助工單數
可靠度 (Reliability)	平均故障間隔時間 (MTBF)；單月無預警當機機率；系統可用性百分比 (4 個 9)
強韌度 (Robustness)	系統崩潰重啟並恢復服務時間 (MTTR)；斷電時資料毀損遺失比率
可移植性 (Portability)	目標平台相依程式碼比例；支援跨平台作業系統 (Linux/Windows/macOS) 數量

觀念檢測：非功能性需求指標量化 (CCQ 3)

客戶提出了一個模糊的非功能性目標：*「訂單結帳系統必須具備極致速度與極高可靠度。」* 下列哪一項敘述正確地將此模糊目標轉化為**可驗證、可測試的工程量化指標**？

- **A.** 99% 的結帳交易回應時間須 ≤ 500 毫秒，且尖峰時段系統可用性須達 $\geq 99.95\%$。
- **B.** 結帳使用者介面應採用滑順流暢的過場動畫，使顧客在心理層面感受最極致的速度。
- **C.** 所有結帳微服務皆須採用記憶體安全程式語言開發，以百分之百確保程式零缺陷執行。
- **D.** 雲端資料庫應於尖峰交易負載攀升時，自動配置無上限的隨機存取記憶體 (RAM)。

課堂活動：Cybercab 非功能性指標量化 (Pair Discussion)

> *情境設定：Elon Musk 提出願景：「Cybercab 必須提供地表最安全的自駕體驗、最順暢的租車流程、以及最毫不費力的自動還車！」身為首席系統工程師，你如何將這些願景轉化為客觀可驗證的 **NFR** 量化指標？*

1. 自駕安全與反應延遲

將「安全自駕」轉化為測試指標：

- **引導思考面向：**
  - 如何以毫秒 (ms) 為單位定義從感測辨識到煞車致動的極限延遲？
  - 在大雨、濃霧、黑夜等極端天候下，這項指標必須如何被保證？
  - 測試規範應要求達到哪一個統計百分位數（例如 99.99% 信心門檻）？

2. 租車與即時派遣流程

將「順暢租車」轉化為測試指標：

- **引導思考面向：**
  - 雲端派遣系統媒合附近可用車輛並發送路線指令需於幾秒內完成 ($\leq N$ 秒)？
  - 租客抵達車側時，手機藍牙 / NFC 感應解鎖車門的最大容許延遲？
  - 乘客上車身分臉部或手機驗證的錯誤拒絕率 (FRR) 上限為何？

3. 還車、車況檢查與結算

將「無痛還車」轉化為測試指標：

- **引導思考面向：**
  - 車內攝影機與超音波感測器檢測遺留物品或垃圾需於幾秒內完成？
  - 允許租客確認完成還車前，車輛電池必須具備之最低保留電量門檻？
  - 系統完成車輛狀態安全核驗、押金解圈與租車扣款確認之時限？

課堂活動：AI 需求提示詞工程 (雙人同儕演練)

> *情境設定：兩人一組，針對相同產品概念進行測試：「一個連結消費者、餐廳與外送員的外送平台 (Food Delivery app)，提供即時外送到府服務。」比較提示詞結構如何根本性改變 AI 規格書品質：*

學生 A：直接未結構化提示詞

缺乏工程指引的直覺式 Prompt：

- **你的提示詞：** *「請為這個外送平台系統產生需求規格書。」*
- **檢驗 AI 產出的結果：**
  - 缺乏邊界、籠統的功能清單列舉。
  - 充斥模糊且不可測的非功能詞彙 (*「極速回應」*、*「高度安全」*、*「介面直覺」*)。
  - 混淆前端使用者操作與後端資料庫細節。
  - 完全遺漏食品衛生、稅務法規等領域需求。

學生 B：多維度 RE 架構提示詞

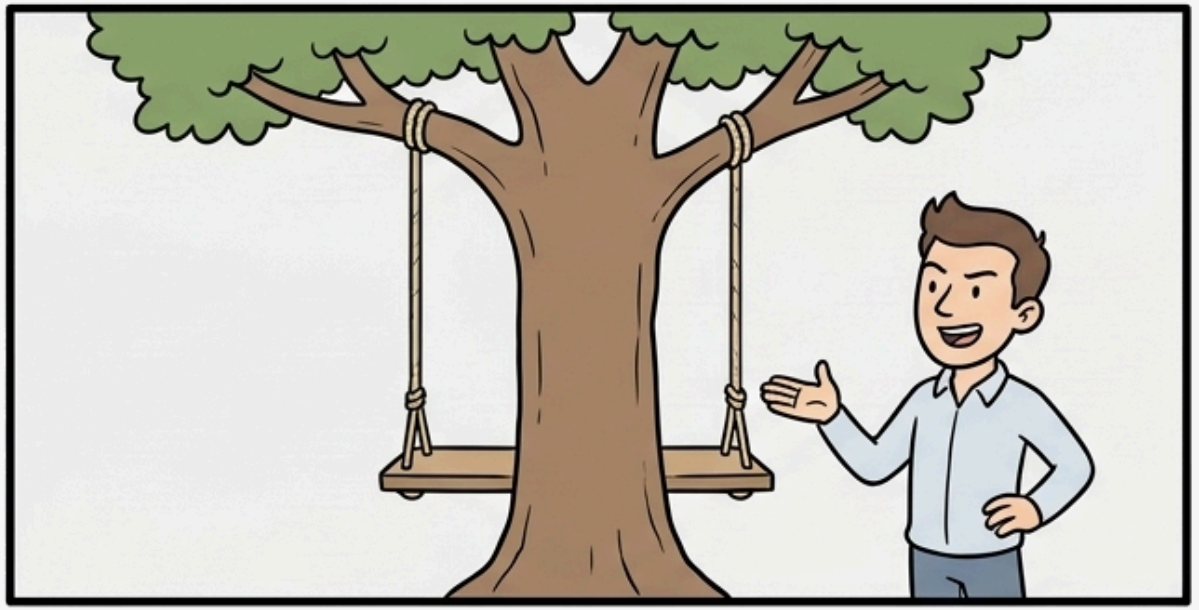
套用本章需求工程框架的專業 Prompt：

- **你的提示詞要求：**
  - **FR vs. NFR：** 明確要求將非功能性需求拆解至可度量指標 ($\leq 500extms$ 延遲、.95%$ 可用率)。
  - **使用者 vs. 系統：** 清楚分離使用者操作目標與系統後端技術合約。
  - **領域規則：** 強制要求納入 PCI-DSS 金流安全與溫控法規。
- **互相比較：** 學生 B 的產出在嚴謹度上有多大躍升？

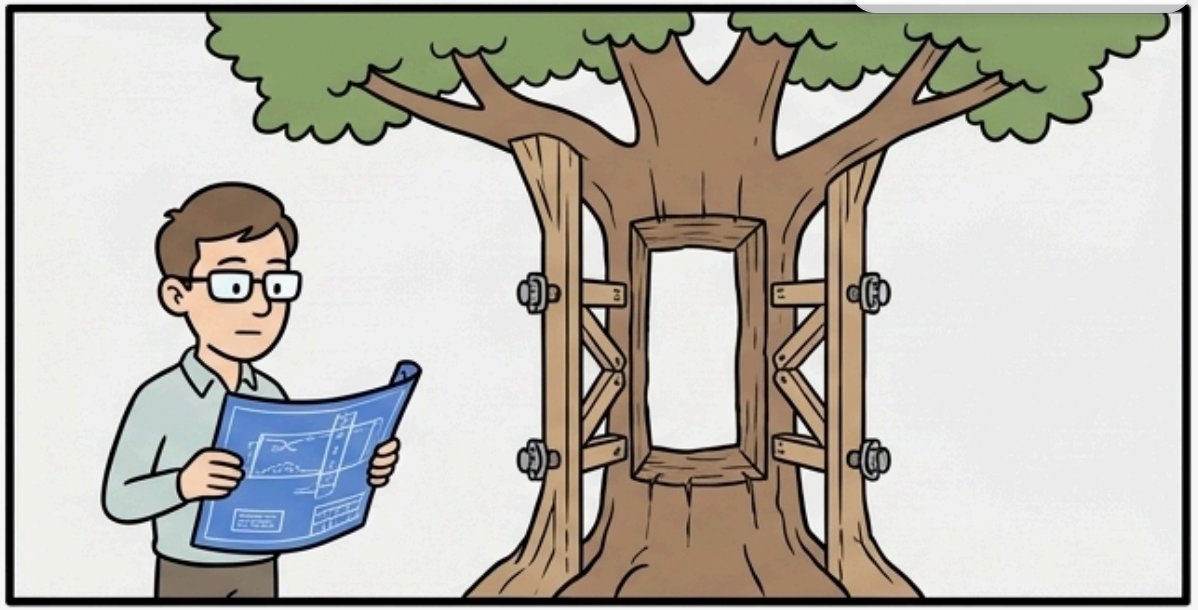
3.4 需求擷取與系統建模 (Elicitation & Modeling)

> *"Customers do not know what they want until you show it to them — elicitation is a collaborative journey of discovery."*
> *「客戶往往在看到具體雛形之前，並不真正清楚自己需要什麼——需求擷取是一場共同探索的協同之旅。」*

What the customer described



How the systems analyst designed it



How the programmer coded it



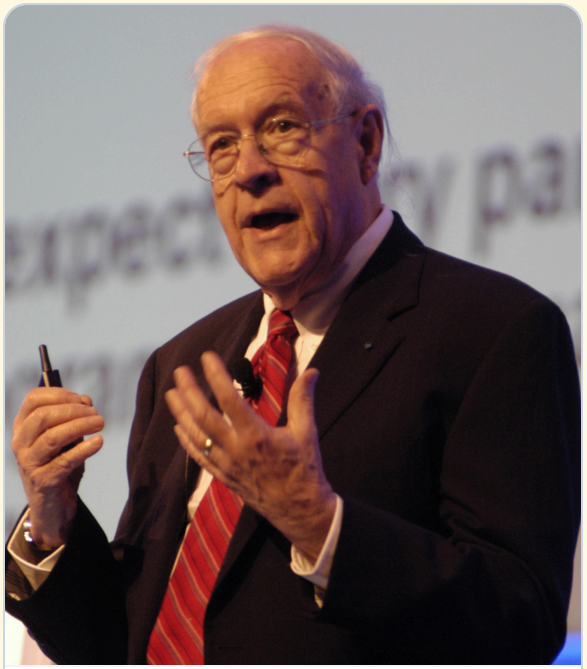
What the customer actually wanted



為何需求擷取如此艱鉅？ (Why Elicitation is Hard)

> *「打造軟體系統最困難的環節，莫過於精確決定究竟要建構什麼。」*
> *— **Fred Brooks**, 沒有銀彈 (No Silver Bullet, 1987)*

- **1. 表達障礙與內隱知識 (Tacit Knowledge):**
  - 領域專家習慣成自然，難以用語言完整描述日常例外情境。
- **2. 解法偏誤與過早錨定 (Solution Bias):**
  - 客戶傾向直接提出具體功能/技術規格，而非業務痛點本身。
- **3. 利害關係人立場衝突 (Conflicting Agendas):**
  - 行銷、資安與財務部門對系統目標有著根本性的相互矛盾。
- **4. 需求易變性與移動目標 (Volatility):**
  - 一旦接觸到可操作的原型系統，利害關係人的期望便會迅速演化。

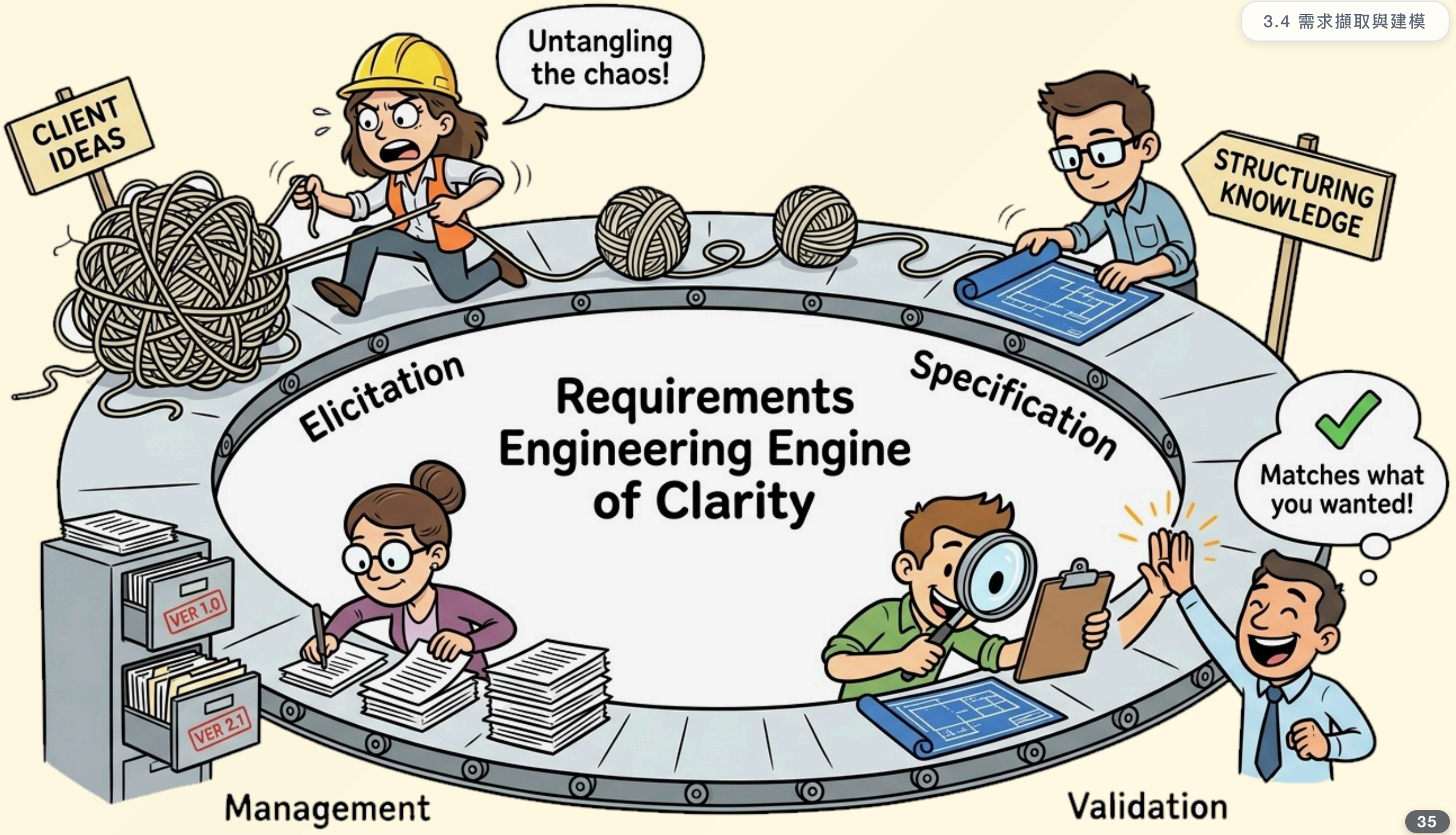


Fred Brooks (1931–2022)
圖靈獎 (1999) / 維基百科

需求擷取與系統建模 (Requirements Elicitation & Modeling)

> *探索、理解、協商並以模型形式具體界定利害關係人需求與系統運作邊界的協同疊代過程。*

- **交織進行的四大工程活動 (Four Interleaved Activities):**
  1. **需求擷取與分析 (Elicitation & Analysis):** 主動與多元利害關係人溝通，挖掘深層業務需求。
  2. **需求規格化 (Specification):** 將探尋所得轉化為結構嚴謹、條理分明的技術文檔或模型。
  3. **需求驗證 (Validation):** 嚴密審查規格內容是否忠實反映客戶真實意圖且具備實作可行性。
  4. **需求變更管理 (Management):** 在軟體生命週期中，制度化地追蹤、審查與管控後續所有變更。
- **螺旋疊代本質 (Spiral & Iterative Nature):**
  - 需求工程**絕非單向一次性的瀑布式線性活動**；它是一條在每個釋出版本中不斷深化認知、持續收斂的動態螺旋。


CLIENT IDEAS
Untangling the chaos!
STRUCTURING KNOWLEDGE
Elicitation
Requirements Engineering Engine of Clarity
Specification
Matches what you wanted!
VER 1.0
VER 2.1
Management
Validation

需求擷取技術 1：訪談與引導式工作坊 (Interviews & Workshops)

- **訪談架構三大類型 (Types of Interviews):**
  - **封閉式訪談 (Closed Interviews):** 預先擬定標準化題目，特別適用於驗證法規檢核清單與既定事實邊界。
  - **開放式訪談 (Open Interviews):** 不設限的探索性對談，旨在挖掘使用者深層痛點、日常困擾與宏觀願景。
  - **半結構化訪談 (Semi-Structured Interviews):** 業界主流標準，掌握核心議題大綱，並保有隨機深入突發痛點的彈性。
- **「未言明的隱含假設」陷阱 (The "Unstated Assumptions" Trap):**
  - 領域專家常將關鍵專業知識視為理所當然而徹底省略（*「這太基本了，全天下誰不知道每區的營業稅率都不一樣？」*）。
- **共同應用程式設計工作坊 (Joint Application Design - JAD):**
  - 將不同立場的利害關係人（醫師、護理師、IT、會計）聚集於同一間會議室，由中立引導師主持，數小時內化解跨部門爭端。

訪談啟發法：原則、迷思與實戰技巧 (Interview Heuristics)

- **核心指導原則 (Core Principles):**
  - **聆聽 80%、發言 20% (Listen 80%, Talk 20%):** 讓利害關係人完整暢談工作痛點，工程師切忌急於插話提供解答。
  - **聚焦「問題本質」而非「解法形式」:** 詢問*「您的業務目標是什麼？」，*而非*「您想要下拉選單還是勾選框？」*
- **常見認知迷思 (Common Myths):**
  - *迷思 1：* 「使用者完全清楚自己需要什麼並能精確表達。」（實務：他們常把表面症狀與根本病因混為一談）。
  - *迷思 2：* 「只要做一次深度訪談就能完成完美規格。」（實務：訪談只是提供最初步的假設線索）。
  - *迷思 3：* 「在訪談中和使用者討論技術架構與資料庫設計。」（實務：充滿專有名詞的技術術語只會讓使用者築起心防）。
- **實戰進階技巧 (Practical Skills):**
  - **連續五次問為什麼 (The "5 Whys" Technique):** 層層剝除表層技術要求，探尋真正的商業核心痛點。
  - **享受沉默力量 (3 秒靜默留白):** 當受訪者語畢時，刻意等待 3 秒鐘，往往能引導出最真實未經修飾的內隱實況。

需求擷取技術 2：民族誌觀察法 (Ethnography & Observation)

> *"What people say they do, and what they actually do, are often entirely different."*
> *「人們聲稱自己做的事情，與他們實際做的事情，往往截然不同。」*

- **直接深入現場所展現的威力 (Direct Observation):**
  - 親身進駐軟體即將運作的真實物理環境（例如：醫院急診室、高頻交易室、物流分揀碼頭）。
- **發掘深層「內隱知識」 (Uncovering Tacit Knowledge):**
  - **內隱知識 (Tacit Knowledge):** 使用者高度熟練、深植日常習慣以致無意識操作的非正式捷徑，在會議中絕不會主動提及。
- **觀察風格與手法 (Observation Styles):**
  - **被動觀察法 (Passive Observation / "壁上蒼蠅"):** 不加干預地靜默觀察自然工作流程，精確辨識作業瓶頸與認知負荷。
  - **情境脈絡探詢 (Contextual Inquiry / 影子跟行):** 在使用者執行真實作業時進行影子跟隨，適時發問（*「為什麼您剛才要把這組代碼抄在黃色便利貼上？」*）。

需求擷取技術 3：情境、使用者故事與故事板 (Scenarios & Stories)

- **具體敘事型情境 (Concrete Narrative Scenarios):**
  - 富含生動細節的時序性走查，模擬真實人物在一般常態與極端危機狀況下的系統操作過程。
- **敏捷使用者故事與驗收準則 (Agile User Stories & Acceptance Criteria):**
  - **經典範式：** *「身為一個 [使用者角色]，我想要 [功能服務]，以便於 [達成商業價值]。」*
  - **Given-When-Then (Gherkin 語法):**
    - *假設 (Given)* 消費者已套用一張有效的滿額折扣碼，
    - *當 (When)* 消費者將符合活動資格的餐點加入購物車並結帳，
    - *那麼 (Then)* 結帳總金額必須折抵 20%，最高折抵上限為 $10。
- **低傳真故事板與紙本原型 (Storyboarding & Paper Prototyping):**
  - 以連環漫畫形式快速勾勒使用者旅程，在未寫一行程式碼前迅速獲取直觀回饋（*「天啊，我居然要點 5 次螢幕才能重新下單！」*）。


Agile User Story
As a Hungry Customer, I want live GPS tracking of my UberEats courier, so that I don't wait in the cold rain!


Uber Eats


9:41
GIVEN courier is within 2 minutes,
WHEN app alerts customer,
THEN food is delivered hot and fresh!


Acceptance Criteria Met
Uber Eats

需求擷取技術 4：文件分析與系統考古 (Document Analysis)

- **挖掘既有營運資產 (Mining Operational Artifacts):**
  - **標準作業程序書 (SOP):** 企業既有官方規章、法規遵循手冊與品管檢核流程。
  - **舊系統原始碼與資料庫綱要:** 透過逆向工程解析上線運作數十年的老舊系統，提煉隱含業務演算法。
  - **表單與試算表考古 (Forms & Spreadsheet Archaeology):**
    - 紙本簽呈表單、報帳收據，以及員工私下使用的龐大 Excel 追蹤試算表。
    - *工程經驗黃金法則：員工活頁簿中手動增設的每一個自訂欄位，都代表現行系統未被滿足的一項核心需求！*
- **「考古陷阱」的防範 (The "Archaeology Trap"):**
  - 必須嚴格區分**不可或缺的本質商業邏輯**（務必完整保留）與**歷史性技術妥協產物**（應予廢除，絕不可盲目照搬數位化）。

需求擷取技術評估矩陣 (Choosing Elicitation Techniques)

技術手法	核心優勢	主要限制	最佳適用時機
訪談法 (Interviews)	挖掘深度大；能建立與客戶的情感信任	易受主觀偏見影響；無法捕捉內隱知識	專案早期探索廣泛目標與建立人脈
工作坊 (JAD)	能高效化解立場衝突；快速凝聚共識	排程困難；強勢人格容易主導全場	多方利害關係人存在嚴重利益矛盾時
民族誌 (Ethnography)	發現真實操作行為與非正式捷徑	極耗費時間與人力；成果較難通用化	複雜高壓、高危險的實體工作現場
故事與情境 (Stories)	具備高度可測性；直接銜接敏捷衝刺待辦	容易見樹不見林，遺漏系統整體非功能限制	定義使用者面向的增量式功能模組
文件考古 (Doc Analysis)	客觀中立；精確還原舊系統遺留業務邏輯	規格文件經常過時失真；容易照單全收	汰換遺留核心系統與高度受監管領域

THE 4 REQUIREMENTS ELICITATION TECHNIQUES



觀念檢測：訪談啟發法與五個為什麼 (CCQ 4)

在需求訪談過程中，一位醫院主管堅持要求：*「我們的臨床資訊系統必須導入區塊鏈帳本來記錄病患的生理生命徵象。」* 此時軟體工程師最應採用的專業訪談啟發法為何？

- **A.** 運用「五個為什麼 (5 Whys)」探詢其背後的資料防竄改與稽核痛點，而非過早鎖定特定技術。
- **B.** 立即針對客戶指定的區塊鏈功能，著手撰寫智慧合約與關聯式資料庫對應綱要。
- **C.** 當場直接駁回該主管的要求，因為非技術利害關係人不得主動指定系統的實作架構。
- **D.** 禮貌性終止本次訪談流程，並將後續需求擷取工作完全轉向被動的工作場所觀察法。

觀念檢測：民族誌與內隱知識挖掘 (CCQ 5)

在需求工程中，當分析急診室或飛航管制等複雜且高壓的運作環境時，為何**民族誌觀察法 (Ethnography / 職場觀察)** 具備無可取代的關鍵價值？

- **A.** 能發掘內隱知識——即使用者已習以為常且視為當然，因而在正式訪談中鮮少主動提及的工作細節與非正式捷徑。
- **B.** 能自動將利害關係人的自然語言需求，在完全無須人工介入的情況下直接編譯為可執行的驗收測試套件。
- **C.** 能完全消除後續的軟體架構設計、關聯式資料庫建模以及同儕程式碼審查 (Code Review) 等開發階段。
- **D.** 能確保所產出的軟體需求規格書在專案的第一個開發衝刺期 (Sprint) 即達成 100% 的數學完全性。

從情境邁向使用案例：貫通擷取與系統建模 (Scenarios to Use Cases)

> *工程師該如何將擷取技術 3 中蒐集到的零散人類故事，系統化收斂為結構嚴謹的系統模型？」*

- **銜接擷取技術 3 (情境與使用者故事):**
  - 需求擷取階段產出了許多真實且具體的感性故事（*「小芬使用過期折價券訂握壽司，結果信用卡扣款失敗」*）。
- **何謂情境 (Scenario)？**
  - 使用者在系統中進行互動的**單一具體執行路徑**（單一敘事序列）。
- **何謂使用案例 (Use Case)？**
  - 一個通用的功能保護傘，匯聚了特定參與者為了達成明確業務目標（如*訂購餐點*）所涉及的**所有相關情境**（包含正常成功路徑，以及所有替代與例外錯誤流程）。
- **為何必須從情境昇華為使用案例？**
  - 情境提供了豐富的同理心脈絡；使用案例則將其梳理為結構化、可供工程實作與編寫完整測試案例的正式功能合約。

發掘使用案例的五步工程法 (Discovering Use Cases)

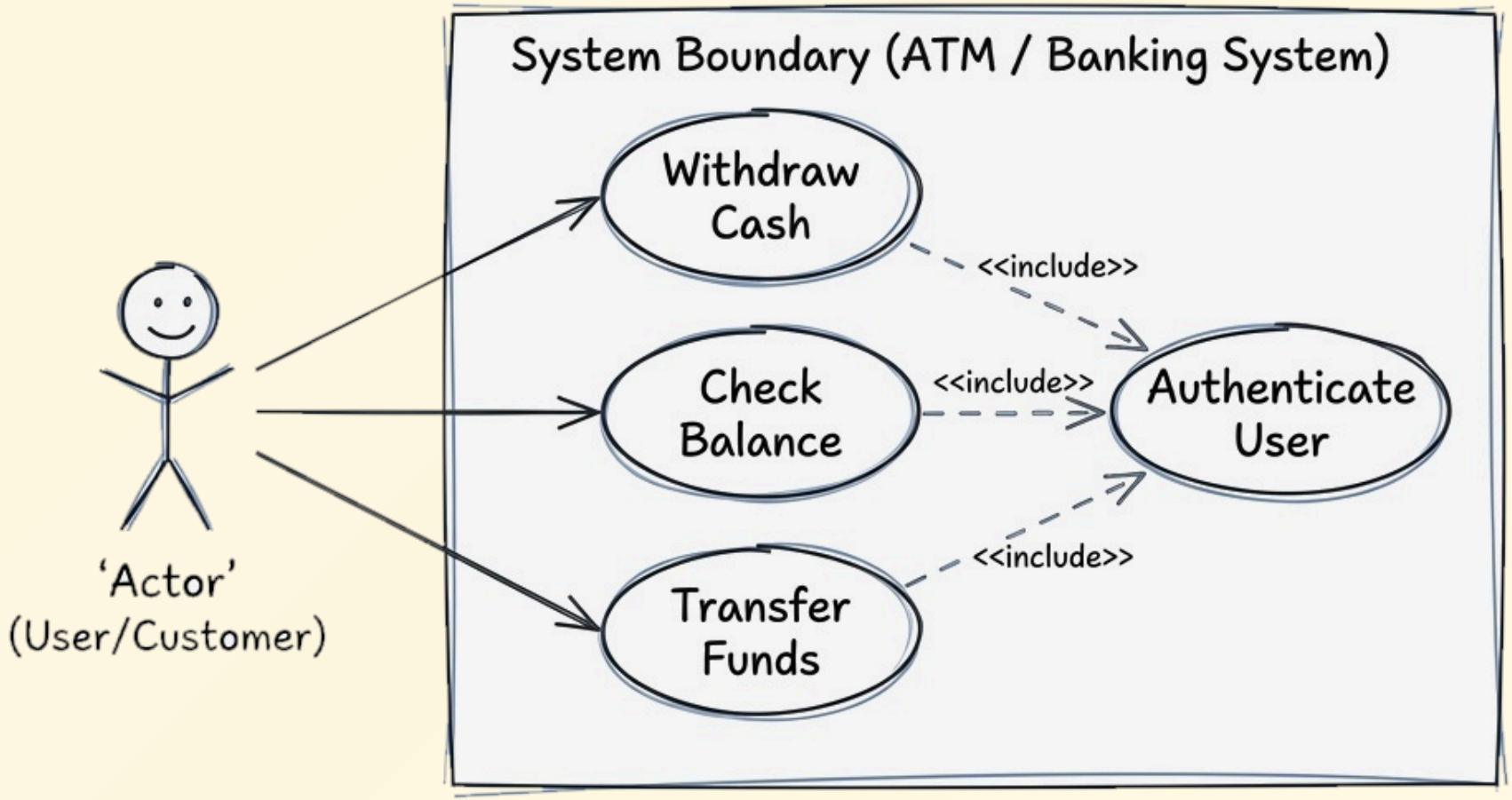
軟體工程師如何系統化地引導利害關係人，在需求擷取中發掘並結構化使用案例？

1. **辨識外部參與者 (Identify External Actors):** 釐清誰或什麼外部實體會直接與系統互動（人類角色、第三方金流閘道、自動化感測器）。
2. **定義參與者的業務目標 (Identify Actor Business Goals):** 針對每位參與者追問：*「他們需要系統交付什麼具體可觀察的價值？」*
3. **歸納情境為使用案例 (Group Scenarios into Use Cases):** 將相關的使用者故事匯聚為動名詞目標（*下單訂餐*、*即時追蹤外送*）。
4. **劃定系統邊界 (Define System Boundaries):** 嚴格標記哪些運算邏輯歸屬本系統範疇，哪些屬於外部服務或人工流程。
5. **提煉通用與條件式邏輯 (Factor Common & Conditional Logic):** 抽取共用必備子程序（`<<include>>`）與邊界條件擴充（`<<extend>>`）。

使用案例圖 UML 記號與語意 (Use Case Diagram Notation)

- **參與者 (Actors):** 與應用系統進行直接互動的外部實體（使用者角色或外部系統）。
- **使用案例 (Use Cases):** 橢圓形圖示，代表能為參與者產出具體可觀察價值的高階功能互動。
- **系統邊界 (System Boundary):** 矩形外框，明確界定本系統實作範圍與外在世界的邊界。
- **關鍵關聯關係 (Relationships):**
  - `<<include>>` **(包含關係):** 基礎使用案例在執行過程中，**必然且無條件**會呼叫並執行的共通子程序（例如：*查詢病歷* 必然包含 *身分驗證*）。
  - `<<extend>>` **(擴充關係):** 僅在特定條件成立或觸發例外時，才會**有條件介入**擴充的延伸行為（例如：*開立管制藥品處方* 擴充自 *開立一般處方*）。

Anatomy of a UML Use Case Diagram



System Boundary (ATM / Banking System)

USE CASE DESCRIPTION: WITHDRAW CASH

METADATA

- Primary Actor: Bank Customer
- Preconditions: ATM online
  - Cash vault loaded
  - Card inserted & validated
- Postconditions: Cash dispensed
  - Account debited
  - Audit log recorded
  - Card returned

MAIN SUCCESS SCENARIO (HAPPY PATH)

1. Customer selects 'Withdraw Cash' and inputs amount ($100)
2. System verifies available account balance with Core Banking API
3. System confirms internal ATM vault has sufficient bills
4. System debits customer account & creates transaction log
5. System initiates cash dispensing mechanism
6. System ejects ATM card and sounds chime
7. Customer removes card; System opens cash dispenser shutter
8. Customer takes cash; System prints transaction receipt

EXTENSIONS & EXCEPTIONS

2a. Insufficient Balance: System displays alert, offers balance check, and aborts
3a. Machine Out of Cash: System alerts customer, limits withdrawal, or aborts
6a. Unclaimed Card: If card not taken within 30s, ATM retracts card for security
8a. Unclaimed Cash: If cash left in tray for 30s, shutter closes and debit is reversed

觀念檢測：使用案例圖與說明書互補性 (CCQ 6)

軟體工程團隊繪製了一份 UML 使用案例圖，標註使用者與「提領現金」使用案例相連。為何工程師除了使用案例圖外，還必須撰寫詳盡的文字型**使用案例說明書 (Use Case Description)**？

- **A.** 圖形僅呈現高階系統範疇；文字說明才能完整定義循序步驟、前置條件、後置條件與例外處理流程。
- **B.** 現行現代網頁瀏覽器若缺乏伴隨的 Markdown 純文字描述，將完全無法正確渲染 UML 向量圖形。
- **C.** 現代編譯器與虛擬機器需要文字型使用案例說明，以便在執行階段預先為角色執行緒配置堆積記憶體。
- **D.** 文字型使用案例說明的核心目的，是將非功能性需求自動轉化為前端圖形使用者介面的高傳真原型。

課堂活動：Cybercab 需求擷取活動規劃 (Pair Discussion)

> ***10 分鐘雙人任務：*** *你們是 Cybercab 首席需求工程師。請於 10 分鐘內在紙上或共享文件產出 **3** 大需求擷取具體行動方針：*

產出 1：利害關係人與痛點 (3m)

挑選 2 類存在利益衝突的群體：

- **行動指引：**
  - 挑選 2 類：如通勤乘客、自用釋出車主、身障行動不便者、或市政交通局。
  - **你的具體產出：** 為各群體歸納 **1 項問卷問不出來的「內隱痛點」**。

產出 2：深度擷取技術匹配 (4m)

為每類群體匹配 1 項最有效的擷取技術：

- **行動指引：**
  - 從 3.4 節技術中選取：實地脈絡觀察法、車室 VR 雛型測試、或焦點工作坊。
  - **你的具體產出：** 清楚論證*為什麼*該技術能挖掘出其內隱需求與情緒摩擦點。

產出 3：利益衝突與仲裁 (3m)

定義並仲裁雙方 1 項直接衝突的需求：

- **行動指引：**
  - 定義 1 項衝突：例如乘客希望隨處路邊上下車 vs. 城市交通安全管制。
  - **你的具體產出：** 運用 **MoSCoW** 優先級框架（Must-Have vs. Won't-Have）判定勝出者並說明工程權衡理由。

3.5 需求驗證與變更管理 (Validation & Change Management)

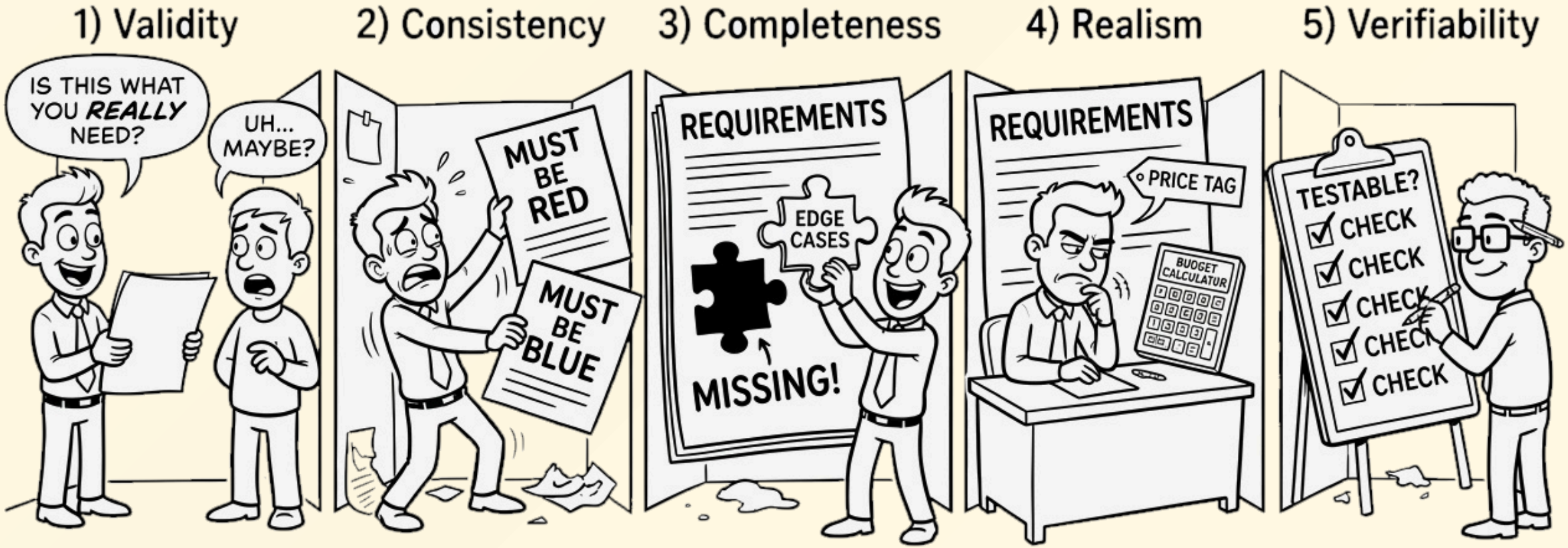
> *"Change is inevitable. In a living system, requirements management is the art of embracing change without descending into chaos."*
> *「變更是不可避免的。在一個具生命力的系統中，需求管理是一門擁抱變更卻不致陷入混亂的藝術。」*

需求驗證與變更管理 (Validation & Change Management)

> *確認需求規格確實反映利害關係人真實意圖，並在軟體整個生命週期中制度化管理規格演進的關鍵品管程序。*

- **後期修正需求缺陷的巨大代價 (High Cost of Late Defects):**
  - 在系統交付上線後才修正需求錯誤，所耗費的工程成本是早期修正的**高達 100 倍**！
- **需求驗證五大檢驗支柱 (5 Requirements Checks):**
  1. **有效性 (Validity):** 需求是否真實反映客戶解決問題的實質需求？
  2. **一致性 (Consistency):** 需求條文之間是否存在前後矛盾或衝突定義？
  3. **完全性 (Completeness):** 是否已完整涵蓋所有系統功能、運作約束與邊界例外？
  4. **現實性 (Realism):** 考量現有預算、時程與既有技術能力，該需求是否真能被實作？
  5. **可驗證性 (Verifiability):** 該需求是否具備客觀明確的量化標準，能被工程團隊客觀測試？

The 5 Pillars of Requirements Validation



課堂活動：五大支柱需求品質審查 (雙人同儕演練)

> ***情境設定：*** *審查以下這段需求草稿：「本系統應即時且輕鬆地完成訂單外送。外送員必須立即接受所有派發的外送任務，但可自由隨意拒絕任何訂單。無論任何情況，即使遭遇強烈颱風，訂單亦必須在 30 分鐘內送達。」*

第一回合：人類同儕審查 (3 分鐘)

與同儕夥伴共同套用五大支柱檢核：

- **有效性 (Validity)：** 這是否真正符合客戶與商業利益？
- **一致性 (Consistency)：** 指出規則中自相矛盾的邏輯漏洞。
- **完整性 (Completeness)：** 標註被遺漏的邊界情境 (餐點打翻、店家關門、退款流程)。
- **現實性 (Realism)：** 颱風天 30 分鐘送達在物理或法律上是否可行？
- **可驗證性 (Verifiability)：** 揪出主觀模糊形容詞 (*「即時」*、*「輕鬆」*)。

第二回合：AI 審查與交叉對比 (3 分鐘)

執行 AI 輔助審查並比對雙方發現：

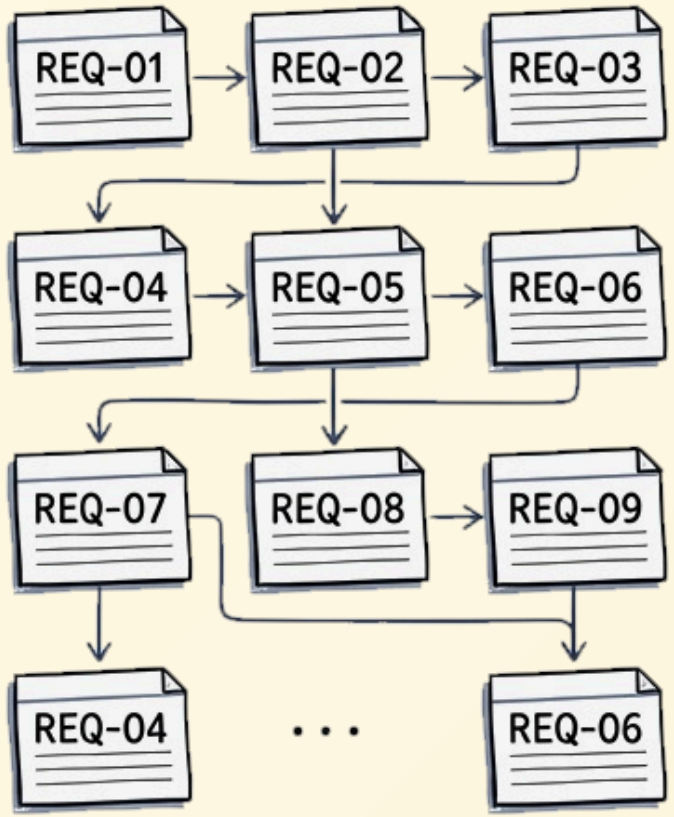
- **提示 LLM：** *「請依據軟體工程五大需求檢驗標準 (有效性、一致性、完整性、現實性、可驗證性) 審查此需求規格：[草稿文字]」*
- **比對人類與 AI 的差異：**
  - **高度共識：** AI 是否也精準抓出矛盾規則與模糊用語？
  - **AI 盲點：** AI 是否忽略了真實世界的勞安法規或人權考量？
  - **AI 強項：** AI 是否發掘出你們兩人未想到的隱藏邊界情況？

需求變更與生命週期治理 (Requirements Change & Management)

- **為何需求必然會變更 (Why Requirements Change):**
  - 商業環境與競爭對手策略劇變、新法規頒布施行、硬體設備更迭，以及使用者操作雛形後認知昇華。
- **需求管理規劃三大核心要素 (Requirements Management Planning):**
  - **唯一編號標籤 (Unique Identification):** 為每一項原子需求標註永久且唯一的識別 ID（例如 `REQ-SEC-012`）。
  - **雙向可追溯性政策 (Traceability Policies):** 建立端到端鏈結：業務目標 → 系統需求 → 架構元件 → 程式原始碼 commit → 測試案例。
  - **變更控制流程 (Change Control Process):** 設立變更控制委員會 (CCB)，在核准變更前嚴格進行影響分析 (Impact Analysis) 與成本效益評估。

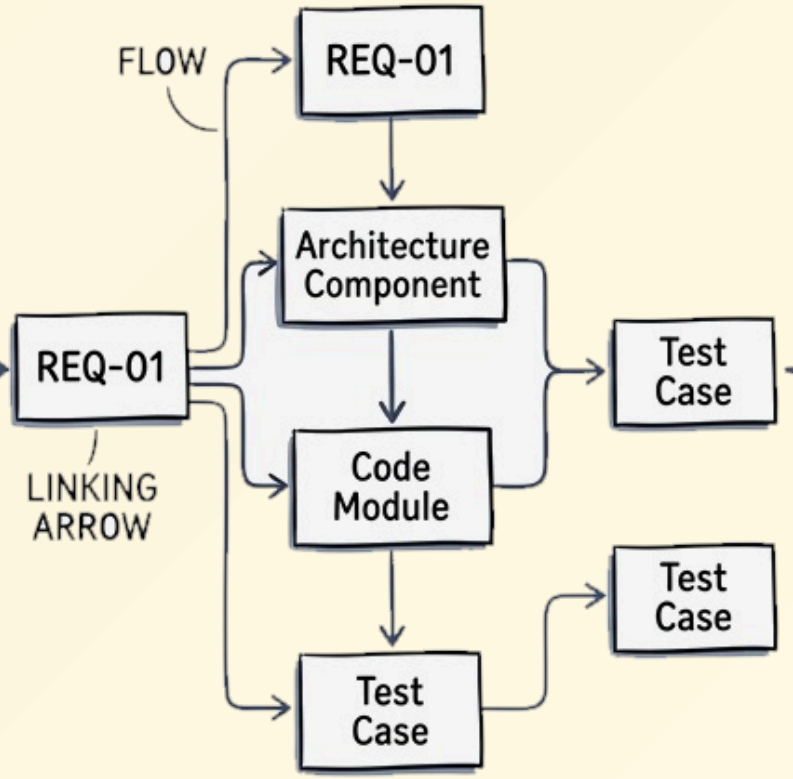
REQUIREMENTS CHANGE MANAGEMENT & TRACEABILITY

UNIQUE ID TAGGING



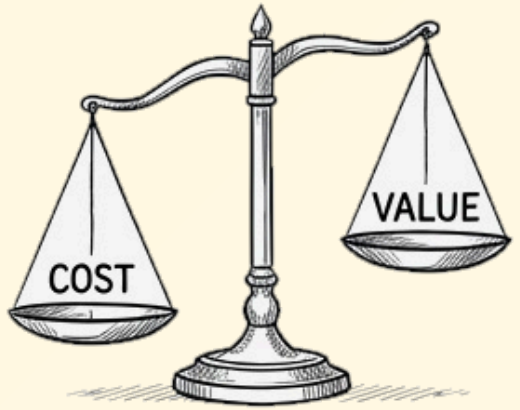
LINILY PRESENT ON
A DIRECT FLOW

TRACEABILITY MATRIX



REQUIREMENTS SAFELY [illegible]
THROUGH CAUSING CHANGES

CHANGE CONTROL BOARD & IMPACT ANALYSIS





IMPACT ANALYSIS

- Potential changes before approved
- Evaluatal changes before approved
- Evaluate changes before approved

觀念檢測：後期需求缺陷的巨大代價 (CCQ 7)

為何在軟體系統交付上線後才修正需求錯誤，其耗費的成本會遠高於在早期開發階段修正？

- **A.** 需求規格文件一旦由客戶簽署核准後，在法律上便具有合約鎖定效力而嚴格禁止進行任何修改。
- **B.** 後期修正必須耗費大量成本重新設計架構、重寫程式碼、重測並重新部署基於錯誤前提建立的連鎖元件。
- **C.** 雲端程式碼儲存庫的自動化編譯器，在軟體發布至正式生產環境後會自動鎖定並阻止程式碼變更。
- **D.** 自動化單元測試套件在程式碼封裝並部署至雲端微服務叢集後，將會完全失去其工程驗證的有效性。



3.6 AI 賦能需求工程 (AI-Assisted Requirements Engineering)

> *"AI is a cognitive amplifier for requirements analysis, but human judgment remains the irreplaceable anchor of accountability."*
> *「AI 是需求分析的認知放大器，但人類工程師的專業判斷，始終是責任歸屬無可取代的唯一錨點。」*

AI 賦能需求工程：現代新前沿 (AI in RE: Overview)

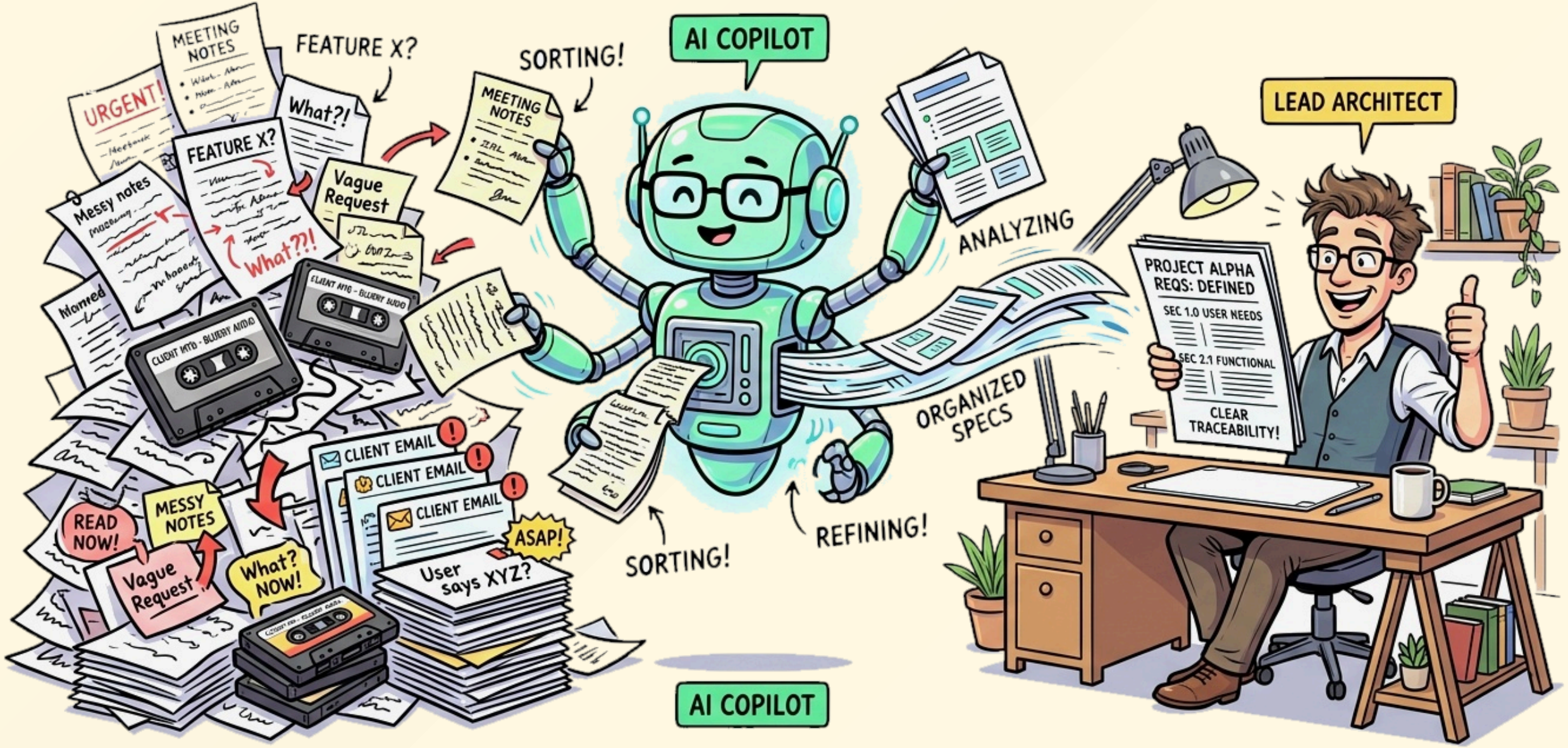
> *善用大型語言模型作為智慧分析副駕駛，架起非正式人類溝通對話通往形式化、可驗證軟體規格的橋樑。*
>
> ”

- **生成式 AI 的典範轉移 (Generative AI Paradigm Shift):**
  - LLM 與生成式 AI 工具在軟體生命週期中扮演**智慧型需求工程副駕駛 (RE Copilot)**。
  - 成功消弭客戶非正式的日常自然語言，與嚴謹形式化的軟體工程產出物之間的鴻溝。
- **核心技術賦能 (Key Capabilities):**
  - 從凌亂雜沓的會議紀錄與訪談逐字稿中，快速提煉出結構化規格。
  - 在數百頁的大型文件中，自動偵測語意條文衝突、被動語態與遺漏的極端邊界條件。
  - 自動生成初始測試案例與符合 Gherkin 語法的 Given-When-Then 驗收準則。

為何 LLM 能有效輔助需求工程？預訓練領域本體論

> *"When you ask an AI to design a food delivery app, it doesn't start from scratch—it already knows the business domain!"*
> *「當你要求 AI 設計外送平台時，它絕不是從一張白紙開始——它早已深諳該商業領域的一切運作架構！」*
> ”

- **預訓練領域知識的無形資產 (Pre-Trained Domain Knowledge):**
  - LLM 在數十億標記中完成了預訓練：涵蓋企業案例、產業白皮書、開源專案程式碼與 ISO/IEEE 國際標準。
  - 模型本身早已具備潛在的**領域本體論 (Domain Ontology)**：
    - **在金融領域：** 它早已熟知帳戶需要交易原子性 (ACID)、多因子認證與反洗錢稽核日誌。
    - **在電商與外送領域：** 它深刻理解多方拆帳分潤、地理圍欄 (Geofencing)、尖峰動態加價與訂單取消退款政策。
- **「即時領域顧問」的工程優勢 (The Instant Domain Advisor):**
  - **發掘內隱需求：** 能主動提點初階工程師經常遺漏的產業標準極端邊界（如：折價券洗單防弊、冷鏈溫控門檻）。
  - **極速工程鷹架：** 數秒內將口語訪談逐字稿轉化為 Given-When-Then 故事與 UML 候選實體。

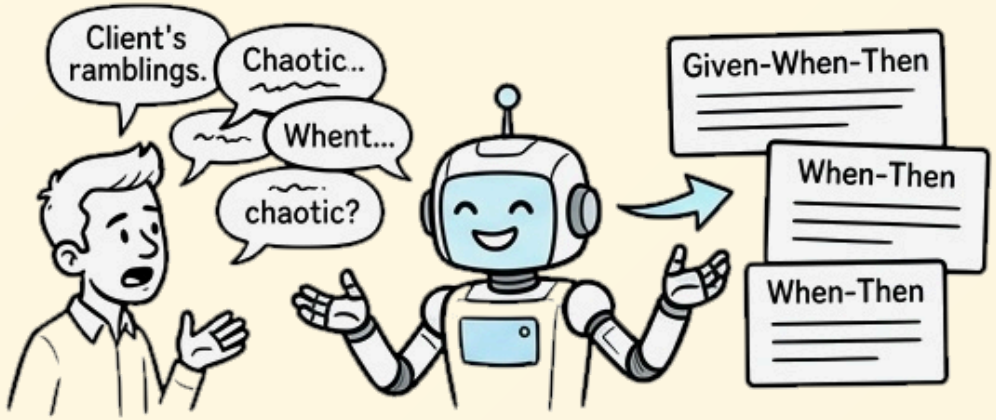

MEETING NOTES
URGENT!
FEATURE X?
What?!
FEATURE X?
What?!!
Messy notes
Vague Request
CLIENT MTG - BLUEBERRY MUFFIN
CLIENT EMAIL
CLIENT EMAIL
CLIENT EMAIL
ASAP!
User says XYZ?
READ NOW!
MESSY NOTES
Vague Request
What? NOW!
SORTING!
MEETING NOTES
AI COPILOT
ANALYZING
SORTING!
REFINING!
ORGANIZED SPECS
LEAD ARCHITECT
PROJECT ALPHA REQS: DEFINED
SEC 1.0 USER NEEDS
SEC 2.1 FUNCTIONAL
CLEAR TRACEABILITY!
AI COPILOT

需求工程中四大核心 AI 實務應用 (4 Core AI Applications in RE)

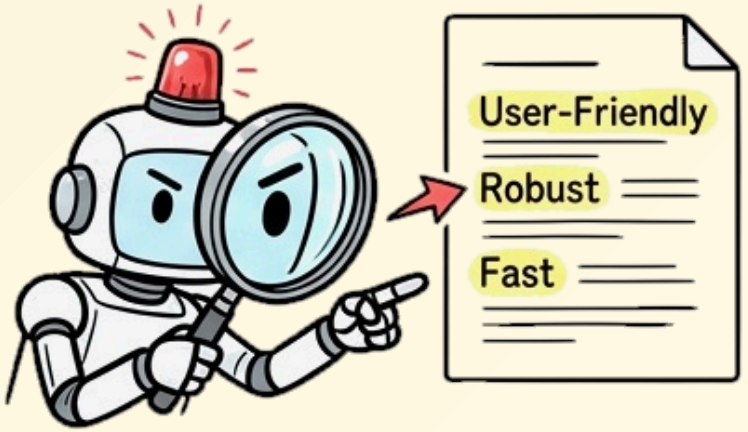
1. **自動化使用者故事與規格草擬 (Automated Story Generation):**
   - 將客戶口語逐字稿，自動轉換為結構嚴謹且符合 `Given-When-Then` 的 Gherkin 驗收準則。
2. **語意歧義與條文衝突檢測 (Ambiguity & Inconsistency Detection):**
   - 快速通讀數百頁規格條文，精確標記出前後矛盾的術語定義與被動語態模糊陳述。
3. **語意關聯之追溯矩陣建置 (Traceability Matrix Construction):**
   - 透過文字向量語意嵌入 (Embeddings)，自動將高階需求映射連結至原始碼模組與單元測試。
4. **合成利害關係人人格模擬 (Synthetic Stakeholder Personas):**
   - 指示 LLM 扮演兒童專科護理師、資安稽核官或眼花長輩，模擬極端情境以發掘被遺漏的邊界需求。

The 4 Core AI Applications in Requirements Engineering

User Story Gen



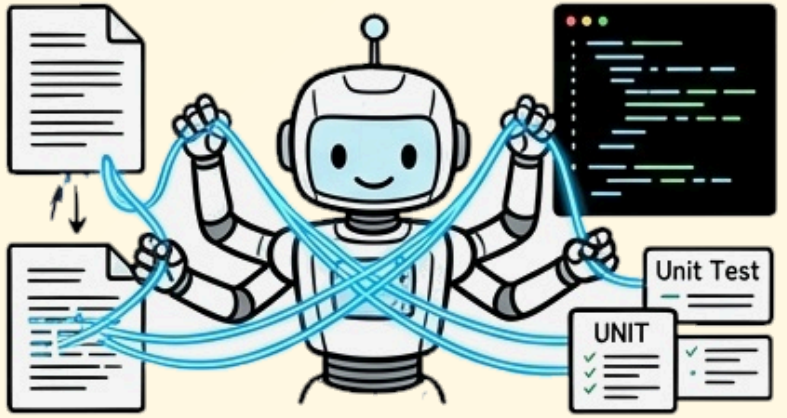
Ambiguity Detective



User Story Gen

Ambiguity Detective





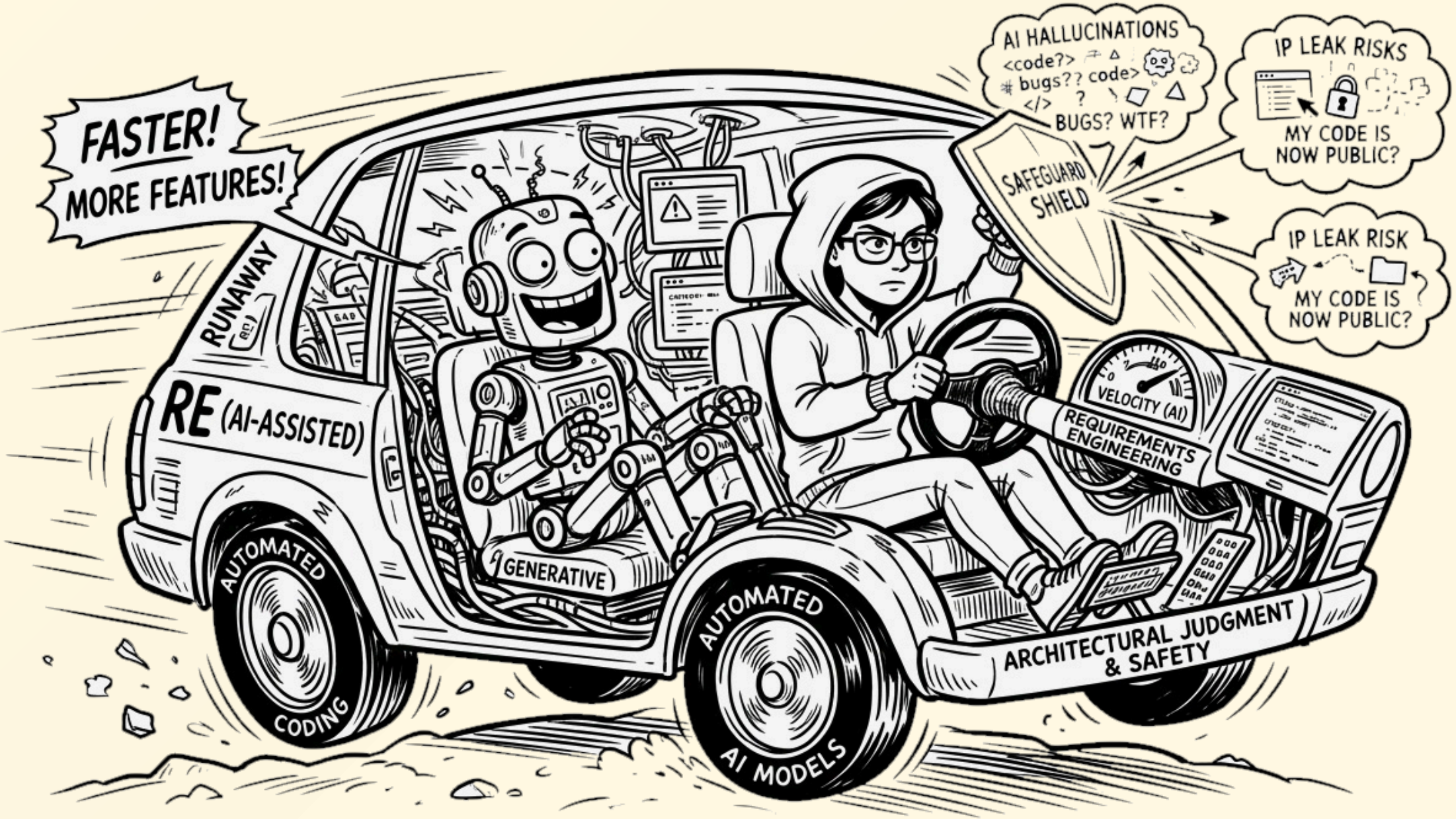
Persona Simulator

Traceability Matrix

人機協同：核心風險與工程最佳實踐 (Human-in-the-Loop)

- **過度依賴無監督 AI 之重大工程風險:**
  - **致命幻覺 (Hallucinations):** 憑空捏造看似合理但完全子虛烏有的業務規則或領域法規。
  - **隱私洩漏與智財侵害 (Privacy & IP Leaks):** 將企業未公開之核心商業機密或病患個資餵入公有雲模型。
  - **歷史偏見放大 (Bias Amplification):** 在需求中強化並放大了訓練資料中原本存在的歷史性偏差。
- **工程黃金法則：人機協同 (Human-in-the-Loop):**
  - **AI 提出建議，人類工程師裁決核准 (AI Proposes; Humans Dispose)。**
  - 唯有人類軟體工程師，承擔整套系統最終的法律責任、職業倫理操守與架構健全性！

HUMAN-IN-THE-LOOP: THE ULTIMATE RESPONSIBILITY IN AI-ASSISTED RE



觀念檢測：生成式 AI 與人機協同檢驗 (CCQ 8)

當軟體工程團隊運用大型語言模型 (LLM) 從利害關係人的訪談逐字稿中自動生成需求規格書時，最需要透過「人機協同 (Human-in-the-Loop)」進行驗證的核心風險是什麼？

- **A.** LLM 可能產生聽起來合理但完全虛構的商業邏輯、遺漏關鍵的邊界約束，或捏造不存在的外部 API 串接。
- **B.** LLM 完全無法產出以 Markdown 清單格式或標準使用者故事 Given-When-Then 語法結構編排的規格文字。
- **C.** LLM 在生成文字時會消耗過量伺服器記憶體，進而導致正式環境中微服務叢集的資料庫發生死結錯誤。
- **D.** LLM 嚴格遵循傳統瀑布式開發流程，因此會堅決拒絕為敏捷疊代衝刺週期生成增量式的需求項目。

3.7 觀念總結與經典文獻 (Recap & References)

> *"Requirements are the bridge between human intention and digital reality."*
> *「需求是連結人類意圖與數位現實的永恆橋樑。」*

重點總結：概念填空測驗 (Fill-in-the-blank Quiz)

測試你對本章核心概念的理解程度：

1. ＿＿ 需求描述系統應提供之具體功能與服務，而 ＿＿ 需求則針對系統之品質與運作施加約束。
2. 以自然語言撰寫、供客戶與非技術利害關係人閱讀的高階敘述稱為 ＿＿ 需求。
3. 嚴謹定義功能細節、作為軟體開發團隊實作與驗收合約基準的規格稱為 ＿＿ 需求。
4. 透過目標-問題-指標 (GQM) 手法，能將模糊的利害關係人願景鍛造為可客觀測試的 ＿＿ 指標。
5. 在 UML 使用案例圖中，基礎案例執行時必然無條件呼叫的共通子程序採用 ＿＿ 關聯性。
6. 在將 AI 導入需求工程時，確保系統規格具備終極責任與可信度的核心工程準則是 ＿＿。

經典文獻與延伸閱讀 (References & Further Reading)

- **經典教科書與國際工程標準:**
  - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 4: Requirements Engineering. Pearson.
  - IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering*.
  - Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
  - Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press.
- **AI 與需求工程前沿實證研究:**
  - Ferrari, A., et al. (2023). "Detecting Ambiguities in Requirements with Large Language Models." *IEEE RE'23*.
  - Arora, C., et al. (2024). "Automating Requirements Elicitation and Analysis with LLMs: An Empirical Evaluation." *ACM TOSEM*.
  - Zhang, H., Rahman, M., & Wang, Y. (2024). "Faithfulness, Hallucination, and Risk in Generative Requirements Engineering." *Empirical Software Engineering (EMSE)*.